

TECHNICAL ARCHITECTURE BRIEF

Pretty Good AI Voicebot Evaluator

Architecture, design decisions, and challenge requirements traceability

Purpose: AI Engineering Challenge submission

Scope: Automated fictional-patient calls to the authorized assessment line

Version: 1.0 - August 20, 2026

Status: Editable submission draft

Source: Pretty Good AI - AI Engineering Challenge (4 pages) and repository review

Scope boundary: All automated calls are restricted in code to +1-805-439-8008. Scenarios use fictional data and are intended only for the authorized assessment.

1. Architecture overview

The system separates prompt generation, live communication, and artifact capture into three independently testable layers. The prompt layer uses a local SQLite FTS5 policy index to retrieve page-aware evidence and build a bounded fictional-patient scenario, or it accepts a fully manual prompt. The communication layer in `realtime_call.py` creates exactly one guarded SignalWire call to the assessment number and opens a bidirectional PCMU media stream to the OpenAI Realtime API. Server-side voice activity detection identifies office turns, while application-level menu filtering, turn settling, response buffering, and playback marks keep the conversation coherent and prevent partial or prerecorded audio from producing premature replies.

The capture layer in `capture_call.py` owns the call-status and recording-status webhooks, timestamped transcripts, authenticated recording download, retries, and per-run artifact directories. This boundary keeps telephony orchestration small while ensuring that each generated scenario, transcript, and MP3 can be traced to a single test case. The design favors a direct Realtime audio bridge over batch speech pipelines for natural turn-taking; local lexical retrieval over hosted vector infrastructure for deterministic setup; and a JSON scenario contract over prompt-only generation so policy evidence, expected safe behavior, and observable failure conditions remain auditable after the call.

2. Challenge requirements traceability

The matrix distinguishes implemented capabilities from submission activities that still require human completion.

2.1 Implemented system capabilities

Area	Challenge requirement	Implementation / evidence	Status
Authorized calling	Call only +1-805-439-8008	Hard-coded destination plus scenario-contract validation; source number is configured separately.	Implemented
Patient simulation	Realistic scenarios across scheduling, refills, questions, and edge cases	RAG generator provides 31 situations across 30 categories; manual prompts are also supported.	Implemented
Recording	Capture both sides in MP3 or OGG	SignalWire dual-channel recording is downloaded as recording.mp3 after finalization.	Implemented
Transcription	Capture both sides of each conversation	Office and played patient turns are timestamped in transcript.txt.	Implemented
Bug discovery	Identify quality or policy failures	Scenario JSON stores evidence, expected safe behavior, and failure conditions for transcript review.	Implemented
Run instructions	Single command after setup	README documents python realtime_call.py --rag after credentials and tunnel setup.	Ready
Architecture	Explain operation and design choices in 1-2 paragraphs	Section 1 contains the required two-paragraph explanation; later sections provide traceability.	Ready

2.2 Submission deliverables and external checks

Area	Challenge requirement	Implementation / evidence	Status
Call evidence	At least 10 complete, 1-3 minute calls	Runtime creates paired artifacts; submission owner must confirm ten complete pairs and call quality.	Complete before submission
Bug report	Document useful issues with call references	Use JSON criteria and transcript timestamps; final consolidated report remains a submission task.	Complete before submission
Loom videos	Walkthrough plus AI-debugging recording	External recordings must be created, made public, and checked against the time/content requirements.	Complete before submission
Repository	Public GitHub repository with no secrets	Code, README, requirements, and .env.example are present; public visibility and secret scan require final confirmation.	Verify before submission
Originating number	Use one source number and submit it in E.164	SIGNALWIRE_FROM_NUMBER supplies one source number for all runs; record it accurately in the form.	Verify before submission

3. Component responsibilities

Component	Role	Responsibility
realtime_call.py	Communication and control	Loads a prompt, validates configuration, starts one guarded call, serves cXML/websocket endpoints, and bridges audio.
scenario_generator/	Prompt and evaluation layer	Selects a diverse policy conflict, retrieves evidence, creates auditable JSON, and validates manual prompt contracts.
rag_pipeline/	Policy knowledge layer	Extracts page-aware PDF chunks and exposes BM25 retrieval through a local SQLite FTS5 index.
capture_call.py	Evidence capture	Persists played patient turns and office transcripts, receives provider webhooks, and downloads the finalized MP3.
SignalWire	Telephony provider	Places the PSTN call, streams PCMU audio, emits call/recording events, and hosts recording media.
OpenAI Realtime API	Conversational patient	Consumes office audio and produces natural spoken fictional-patient responses under the selected prompt.

4. Runtime and data flow

1. At startup, the operator chooses RAG mode or supplies a complete manual patient prompt.
2. RAG mode excludes recently used topics, retrieves relevant policy chunks, and saves a uniquely identified JSON scenario under generated_prompt/.
3. The scenario contract validates the fictional-data marker and authorized destination before its patient_prompt enters the communication layer.
4. SignalWire places one outbound call and requests /cxml, which instructs it to open a bidirectional PCMU stream to /media-stream.
5. The application forwards office audio to OpenAI Realtime, filters recorded menus and incomplete turns, and returns complete patient utterances to SignalWire.
6. Playback acknowledgements determine which generated patient turns are committed to the transcript; office turns are transcribed from input audio.
7. SignalWire sends completion webhooks. The capture layer returns HTTP 204 immediately, downloads the MP3 asynchronously with retries, and reports both artifacts ready.

5. Key design decisions and tradeoffs

Decision	Why selected	Tradeoff
Realtime audio bridge	Produces natural latency and turn-taking, which the challenge evaluates before code quality.	Requires websocket state, audio buffering, VAD tuning, and careful interruption handling.
SignalWire compatibility API	Combines outbound calling, bidirectional streaming, status callbacks, and recordings.	Requires a public HTTPS tunnel and provider credentials during local development.
Local SQLite FTS5 retrieval	Keeps ingestion reproducible, fast, inexpensive, and free of hosted vector-database setup.	Lexical BM25 search has weaker semantic matching than an embedding-based retriever.
Layered JSON contract	Preserves prompt, evidence, expected behavior, and failure criteria for repeatable analysis.	Creates more artifacts and schema maintenance than directly interpolating a prompt.
Playback-confirmed transcript	Records only patient speech acknowledged as played to the remote agent.	Adds mark tracking and buffering complexity but avoids logging unheard generated text.
Hard-coded destination guard	Prevents accidental calls outside the assessment scope.	Intentionally limits reuse as a general-purpose calling platform.

6. Reliability, safety, and evidence quality

- The destination is fixed to the authorized assessment number, and the configured SignalWire source number cannot equal that destination.
- API keys remain in .env; .env.example documents required variables without credentials.
- Generated scenarios require fictional data and impose bounded attempts, final-outcome stopping rules, and no fabricated emergencies.
- Recorded menus and short transcript fragments do not trigger the patient; a settling window reduces false end-of-turn responses.
- Dual-channel recording, playback marks, timestamps, provider status monitoring, and download retries improve evidence completeness.
- Ctrl+T provides an operator-controlled early stop while preserving time for recording finalization.

Not a production healthcare system: The architecture is an assessment harness. It is not designed or represented as a HIPAA production deployment, a clinical decision system, or a tool for real patient data.

7. Verification and submission readiness

Automated checks validate scenario classification, diversity history, destination enforcement, prompt contracts, ingestion, chunking, and retrieval. These tests do not place phone calls. Runtime evidence still requires listening to each recording and reviewing its transcript because conversational quality is the challenge's first evaluation gate.

- Run scenario tests: `python -m unittest discover -s scenario_generator -p 'test_*.py'`
- Run ingestion tests: `python -m unittest discover -s rag_pipeline -p 'test_*.py'`
- For every submitted call, confirm coherent pacing, sensible turn-taking, clear audio, active steering, transcript completeness, and a matching MP3.
- Tie every reported bug to a transcript filename, timestamp, severity, actual behavior, expected behavior, and explanation of impact.

8. Remaining submission actions

- Confirm at least ten output directories contain both a complete transcript and a playable MP3 or OGG recording.
- Select the strongest substantive failures and produce the final bug report with evidence references.
- Record and publish the project walkthrough (maximum three minutes) with webcam enabled.
- Record and publish the separate AI-assisted debugging walkthrough.
- Confirm repository visibility is public, run a secret scan, and verify .env is not committed.
- Submit the one originating phone number in E.164 format and verify it matches every assessment call.